CS791 Assignment1 Team-MSD

Deeksha Koul (24D0365): FHMM
Madhav Kotecha (24M0833): LBP
Smita Gautam (24M0845): Turbocode

September 1st, 2025

1 Inference on Factorial Hidden Markov Models (FHMMs)

Given M chains(factors), each chain i has its own state S_t^i at time t (K possible states). Parameters given for every chain: transitional probabilities i.e $P(S_t^i|S_{t-1}^i)$, emission probabilities $P(O_t|S_t^i)$ and initial probabilities $P(S_1^i)$.

1.1 Compute normalization constant *Z*

Z is defined as

$$Z = \sum_{S_{1:T}} \left(\prod_{t=1}^{T} \psi_t(S_t) \right) \left(\prod_{i=1}^{M} \prod_{t=2}^{T} T^i(S_{t-1}^i, S_t^i) \right)$$

where T^i is the transitional probability of factor i and $\psi_t(S_t) = P(O_t|S_t)$ as emission potential. The S_t here is the joint state tuple from all the chains at timestep t, hence S_t^i is the value of chain i at timestep t.

Forward pass introduction:
$\alpha_t(S) = P(O_{1:t}, S_t = S)$ where 'S' is joint state tuple value from all the chains at timestep t. Therefore, if we sum over all the possible joint states we have $\sum_s \alpha_t(S) = P(O_{1:t})$. For the final timestep: $\sum_S \alpha_T(S) = P(O_{1:T})$. So, we have to do a forward pass to get the α_T over all joint states and then sum to get $P(O_{1:T})$. By the definition of Z from above, it implies that it is enough to get $P(O_{1:T})$, hence calculate α_T for all joint states.
Recursion in forward pass:

$$\alpha_t(S) = \psi_t(S) \sum_{S'} \alpha_{t-1}(S')\, T(S', S),$$

where at t=1 we have $\alpha_1(S) = \pi(S)\, \psi_1(S)$, and $\pi(S)$ is the prior probability of the initial joint state S at t=1, defined as $\pi(S) = \prod_{i=1}^{M} P(S_1^i)$.

1.2 Compute Marginals

For any chain i, posterior marginal can be calculated as:

$$P(S_t^{(i)} = s_i \mid O_{1:T}) = \sum_{S:S[i]=s_i} \frac{\alpha_t(S)\beta_t(S)}{Z}.$$

where the sum is over all the possible joint states from all the chains. For this, we already calculated forward($\alpha's$) and Z, we need to do a backward pass to calculate: $\beta_t(s) = P(O_{t+1:T} \mid S_t = S)$.
Recursion in backward pass:

$$\beta_t(S) = \sum_{S'} T(S, S')\, \psi_{t+1}(S')\, \beta_{t+1}(S'),$$

where $\beta_T(S) = 1$ for all joint states S, $T(S, S') = \prod_{i=1}^{M} T^i(S^i, S'^i)$ is the joint transition probability from S to S', and $\psi_{t+1}(S') = P(O_{t+1} \mid S')$ is the emission potential at time $t + 1$.

1.3 Compute top-k assignment and probabilities

Finding the top-k most probable sequence of assignments over hidden states of all M chains across sequence length(T). A complete assignment of hidden states can be represented as -

$$S_{1:T} = \big((S_1^1, \ldots, S_1^M),\ (S_2^1, \ldots, S_2^M),\ \ldots,\ (S_T^1, \ldots, S_T^M)\big).$$

$$P(S_{1:T}, O_{1:T}) = \pi(S_1)\, \psi_1(S_1) \prod_{t=2}^{T} \Big[T(S_{t-1}, S_t)\, \psi_t(S_t)\Big],$$

where $\pi(S_1)$ is the prior over the initial joint state, $T(S_{t-1}, S_t)$ is the joint state transition probability and $\psi_t(S_t) = P(O_t \mid S_t)$ is the emission potential.
The maximum a posteriori (MAP) assignment is:

$$S^*_{1:T} = \arg\max_{S_{1:T}} P(S_{1:T} \mid O_{1:T}),$$

which is equivalent to maximizing $P(S_{1:T}, O_{1:T})$ as Z is a normalizing constant.

1.4 Computational Complexity

The inference algorithm for a Factorial Hidden Markov Model runs in

$$O\big(T\, M\, K^{M+1}\big)$$

where T is the sequence length, M the number of independent Markov chains, and K the number of states per chain. At each of the time steps T, all K^M joint states are processed, perform M message-passing operations, with each costing $O(K^{M+1})$ due to clique marginals over $M + 1$ variables.

2 Loopy Belief Propagation

Loopy Belief Propagation (LBP) is an approximate inference algorithm that applies sum-product message-passing equations to graphs with cycles. In a Factorial Hidden Markov Model (FHMM), the factor graph has two types of factors: transition factors linking each hidden chain over time, and emission factors coupling all chains at each time step to the observation. LBP iteratively exchanges two kinds of messages:

- *Factor-to-variable messages* $m_{a\to i}(x_i)$ summarize how factor a influences variable i.
- *Variable-to-factor messages* $m_{i\to a}(x_i)$ summarize the product of all other influences on variable i when sending to factor a.

2.1 Factor-to-Variable

1. Compute $m_{a\to i}(x_i)$ from factor a (with scope X_a) to a neighbor variable $X_i \in X_a$.
2. Gather all incoming messages to factor a from each neighbor $X_j \in \mathcal{N}(a) \setminus \{i\}$ $m_{j\to a}(x_j)$
3. Form the integrand by multiplying the factor potential $\phi_a(X_a)$ with the product of incoming messages over all neighbors except i:

$$\phi_a(X_a) \prod_{j\in\mathcal{N}(a)\setminus\{i\}} m_{j\to a}(x_j).$$

4. Marginalize (sum-product) over all variables in $X_a \setminus \{X_i\}$ to obtain an *unnormalized* vector over x_i:

$$\tilde{m}_{a\to i}(x_i) = \sum_{X_a\setminus\{x_i\}} \phi_a(X_a) \prod_{j\neq i} m_{j\to a}(x_j).$$

5. Normalize it:

$$m_{a\to i}(x_i) \;=\; \frac{\tilde{m}_{a\to i}(x_i)}{\sum_{x_i} \tilde{m}_{a\to i}(x_i)}.$$

2.2 Variable-to-Factor

1. Compute $m_{i\to a}(x_i)$ by elementwise multiplying all incoming factor-to-variable messages to variable i from every neighbor $c \in \mathcal{N}(i)\setminus\{a\}$: $m_{i\to a}(x_i) = \prod_{c\in\mathcal{N}(i)\setminus\{a\}} m_{c\to i}(x_i)$.
2. Retrieve each incoming factor-to-variable message vector $m_{c\to i}(x_i)$ for all $c \in \mathcal{N}(i) \setminus \{a\}$. Each vector has length $|X_i|$ and is nonnegative.

3. Normalize $m_{i\to a}$ to avoid underflow:

$$m_{i\to a}(x_i) \;=\; \frac{m_{i\to a}(x_i)}{\sum_{x_i} m_{i\to a}(x_i)}.$$

Further stopping criterion: beliefs can be calculated at each variable i by

$$b_i(x_i) \;\propto \prod_{a\in\mathcal{N}(i)} m_{a\to i}(x_i),$$

and iterations stop when beliefs change below a threshold (Bethe approximation fixed-point).

Time Complexity

Each iteration of loopy belief propagation on an FHMM with F factors, K states, and time steps T costs $O(F\,T\,K^F)$, dominated by summing over K^{F-1} joint emissions per factor. Empirically converging in constant iterations yields overall $O(F\,T\,K^F)$

3 Turbo Codes

3.1 Generating the trellis

Overview. Consider a Rate-$\frac{1}{2}$ Recursive Systematic Convolutional (RSC) encoder with constraint length K. The encoder's state at any time is fully specified by the last $m = K-1$ memory bits. At each time step t, the encoder receives an input bit $u_t \in \{0,1\}$, updates the shift register, applies the feedback function, and generates a parity bit using the feedforward generator.

Consequently, each transition in the trellis is deterministic:

$$(s, u) \mapsto (s', p),$$

where s and s' denote the encoder states (integers in the range $0, \ldots, 2^m - 1$), and $p \in \{0,1\}$ represents the parity bit produced by the transition.

Implementation Details (as in `turbocodes_tester.encoder`). We represent the shift register as a list `reg = [oldest, ..., newest]`, aligning the generator such that its least significant bit (LSB) corresponds to `reg[0]` (the oldest bit) and its most significant bit (MSB) corresponds to the highest-order register bit. For each encoder `state` s and input bit u, the trellis transition is computed as follows:

1. Extract the prefix `reg_prefix` from s, which contains the m older bits of the register.
2. Form the full register: `reg = reg_prefix + [u]`.

3. Calculate the feedback bit: $fb = \bigoplus_i (\text{generator2\_bit}_i \wedge \texttt{reg}[i])$.

4. Update the register by replacing the newest bit with the feedback: `reg_mod = reg[:-1] + [fb]`.

5. Compute the parity bit using the feedforward generator: $p = \bigoplus_i (\text{generator1\_bit}_i \wedge \texttt{reg\_mod}[i])$.

6. Determine the next encoder state s' from the last m bits of `reg_mod`.

Pseudocode

```
function generate_trellis(generator1, generator2, K):
  m = K - 1
  num_states = 1 << m
  mask = (1 << m) - 1
  trans = dict()  # trans[s][u] = (next_state, parity)
  for s in 0 .. num_states-1:
    reg_prefix = bits_of_state(s, m)   # [oldest .. newest-1]
    for u in {0,1}:
      reg = reg_prefix + [u]           # [oldest .. newest]
      fb = xor_from_generator(reg, generator2)
      reg_mod = reg[:-1] + [fb]
      p = xor_from_generator(reg_mod, generator1)
      next_state = bits_to_state(reg_mod[1:]) & mask
      trans[s][u] = (next_state, p)
  return {"num_states": num_states, "trans": trans, "start_state":0, "end_state":0}
```

3.2 Viterbi decoding

Objective. Given the noisy observations of the systematic symbols along with a single parity stream, our goal is to identify the input bit sequence $\hat{u}_{1:T}$ that maximizes the posterior probability $P(u_{1:T} \mid y_{1:T})$. This is equivalent to finding the most likely path through the trellis, also known as the maximum a posteriori (MAP) path.

Log-domain dynamic programming. To prevent numerical underflow, we implement the Viterbi algorithm in the log-probability domain:

$$\text{score}_{t+1}(s') = \max_{s,u \,:\, \text{trans}(s,u)=(s',p)} \big(\text{score}_t(s) + \log P(y_t \mid u, p)\big),$$

where $P(y_t \mid u, p)$ denotes the probability of observing the noisy symbols at time t given the transmitted bits (u, p). We also maintain a backpointer for each (t, s') in order to reconstruct the path that achieves this maximum.

Implementation details.

- At each transition, compute the emission probability as

  `emis = probability_matrix[u][sys_obs] * probability_matrix[p][par_obs].`

- If `emis > 0`, add log(`emis`) to the previous score, otherwise use $-\infty$ (impossible transition).
- Maintain two arrays: `prev_scores` and `cur_scores` of size S, and `backptr[t][s]`.
- At the end choose $\arg\max_s \text{score}_T(s)$ and backtrack.

Pseudocode (viterbi).

```
function viterbi(systematic, parity, prob_matrix, trellis):
  S = trellis.num_states
  prev_scores = [-inf]*S; prev_scores[start_state]=0
  for t in 0..T-1:
    cur_scores = [-inf]*S
    for s in 0..S-1:
      if prev_scores[s] is -inf: continue
      for u in {0,1}:
        ns, p = trellis.trans[s][u]
        emis = prob_matrix[u][sys_obs] * prob_matrix[p][par_obs]
        score = prev_scores[s] + (math.log(emis) if emis>0 else -inf)
        if score > cur_scores[ns]:
          cur_scores[ns]=score; backptr[t][ns]=(s,u)
    prev_scores = cur_scores
  final_state = argmax(prev_scores)
  reconstruct bits via backpointers
  return bitstring
```

Time and space complexity.

- Let T denote the sequence length, $S = 2^m$ the number of states, and $U = 2$ the number of possible inputs.
- Time complexity is $O(T \cdot S \cdot U)$; since $U = 2$ in our case, this simplifies to $O(T \cdot S)$.
- Space complexity is dominated by the backpointer array, requiring $O(T \cdot S)$ to store the full path, plus $O(S)$ for the score arrays (only two consecutive time layers are stored at a time).
- **Implemented optimizations:** using log-domain arithmetic, skipping states with $-\infty$ scores, and relying on arrays instead of dictionaries to improve efficiency.

3.3 BCJR (forward–backward) and Turbo decoding

BCJR per branch. The BCJR algorithm estimates, for each time step t:

$$P(u_t = b \mid y_{1:T}), \quad b \in \{0, 1\}.$$

This is done using the trellis transitions and the following key quantities:

Local transition weight (γ):

$$\gamma_t(s, u) = P(y_t \mid \text{transition } (s, u)) \cdot P(u_t = u),$$

where $P(u_t = u)$ denotes the a priori probability (initially uniform, 0.5, and updated iteratively in Turbo decoding).

Forward recursion (α):

$$\alpha_0(s) = \mathbf{1}\{s = \text{start}\}, \qquad \alpha_{t+1}(s') = \sum_{s,u:\text{trans}(s,u)=(s',\cdot)} \alpha_t(s)\, \gamma_t(s, u).$$

Backward recursion (β):

$$\beta_T(s) = \mathbf{1}\{s = \text{end}\}, \qquad \beta_t(s) = \sum_u \gamma_t(s, u)\, \beta_{t+1}(s').$$

Per-bit posterior probability:

$$P(u_t = b \mid y) \propto \sum_{s:\text{trans}(s,b)=(s',\cdot)} \alpha_t(s)\, \gamma_t(s, b)\, \beta_{t+1}(s').$$

We normalize to get probabilities.

Implementation details.

- Precompute the local transition weights as `gamma[t][s][u]` = emission $\times$ a priori, adding a small ϵ to maintain numerical stability.
- Perform the forward recursion to compute `alpha`, normalizing at each time step to prevent underflow.
- Perform the backward recursion to compute `beta`, applying the same normalization strategy.
- Calculate the per-bit posterior probabilities by summing the contributions from all relevant states at each time step t.

Turbo (loopy belief propagation) loop. The turbo decoder alternates between two single-branch BCJR decoders as follows:

1. **Decoder 1:** Execute BCJR on the original systematic and parity-1 bits, using the current a priori probabilities $\text{apr}^{(k)}$.

2. Compute the posterior probabilities $\text{post}^{(1)}$ from Decoder 1.

3. Determine the **extrinsic information** for Decoder 1 as

$$\text{ext}_t^{(1)}(b) \;=\; \frac{\text{post}_t^{(1)}(b)}{\max(\text{apr}_t^{(k)}(b), \epsilon)},$$

then normalize so that $\text{ext}_t^{(1)}(0) + \text{ext}_t^{(1)}(1) = 1$ at each time step t.

4. Interleave $\text{ext}^{(1)}$ according to the known permutation π to generate the a priori input for Decoder 2:

$$\text{apr}_\pi^{(k)}(t, b) \;=\; \text{ext}_{\pi(t)}^{(1)}(b).$$

5. **Decoder 2:** Run BCJR on the permuted systematic and parity-2 bits, using $\text{apr}_\pi^{(k)}$.

6. Compute the extrinsic information for Decoder 2 as

$$\text{ext}_t^{(2)}(b) \;=\; \frac{\text{post}_t^{(2)}(b)}{\max(\text{apr}_\pi^{(k)}(t, b), \epsilon)},$$

normalize, and then apply the *inverse permutation* π^{-1} to map the results back to the original time order:

$$\text{apr}_t^{(k+1)}(b) \;=\; \text{ext}_{\pi^{-1}(t)}^{(2)}(b).$$

7. Repeat the process for a fixed number of iterations (default 8 in our implementation) or until a convergence criterion is met (see Section 3.3).

Stopping conditions and convergence

– In the implementation we use a fixed number of iterations (default 8).

– Additionally, we implemented a dynamic stopping criterion: after each iteration k, we compare the updated a priori probabilities for bit 1 with those from the previous iteration. If the maximum absolute change across all time steps t falls below a threshold δ, decoding stops early:

$$\max_t \left| \text{apr}_t^{(k+1)}(1) - \text{apr}_t^{(k)}(1) \right| < \delta.$$

– This avoids unnecessary iterations when the algorithm has already converged. If the criterion is never met, decoding falls back to the fixed iteration limit.

Complexity of BCJR and Turbo.

- A single BCJR decoding pass has a time complexity of $O(T \cdot S \cdot U)$, where T is the block length, S is the number of trellis states, and U is the number of possible input symbols. For our binary case, $U = 2$, so this reduces to $O(T \cdot S)$.
- Each Turbo iteration consists of two BCJR runs, resulting in $O(2 \cdot T \cdot S)$ per iteration. For I iterations, the overall complexity is $O(I \cdot T \cdot S)$.
- Space usage is dominated by the forward and backward arrays (α and β) in BCJR, requiring $O(T \cdot S)$ memory. For $m = 2$ (constraint length $K = 3$), $S = 4$, so the memory footprint remains small.

Notes on combining outputs. After completing the final iteration, the decoder applies the inverse permutation π^{-1} to the posteriors from Decoder 2, then averages them with the posteriors from Decoder 1 for each bit:

$$\text{final}_t(b) = \tfrac{1}{2}\Big(\text{post}_t^{(1)}(b) \;+\; \text{post}_{\pi^{-1}(t)}^{(2)}(b)\Big).$$

A hard decision is made by selecting the more probable bit at each time step t.

3.4 Comparing results and BER estimation

Bit Error Rate estimation. For N testcases of length T (here $T = 1000$), the Bit Error Rate (BER) for each decoding algorithm (Viterbi, BCJR, Turbo) is computed as

$$\text{BER} = \frac{1}{N \cdot T} \sum_{i=1}^{N} \text{errors}_i,$$

where errors_i denotes the number of bit mismatches in testcase i.

We evaluated $N = 100$ random testcases and report the estimated BERs in Table 1.

Decoder	Mean BER	Std. Dev.	Min BER	Max BER
Viterbi	0.0370	0.0239	0.0020	0.0840
BCJR	0.0344	0.0207	0.0030	0.0780
Turbo	0.0020	0.0018	0.0000	0.0070

Table 1: Bit Error Rate (BER) over $N = 100$ testcases of length $T = 1000$.

Comparison and observations.

- Viterbi decoding achieved a mean BER of approximately 3.7%, with some variation due to noise (standard deviation 2.4%).

- A single-pass BCJR decoder performed slightly better, yielding a mean BER of 3.4%.
- Turbo decoding with 8 iterations reduced the BER dramatically to around 0.2%, nearly an order of magnitude lower than Viterbi or BCJR.
- For Turbo, the minimum BER observed was 0.0 (some testcases were decoded perfectly), while the maximum remained below 0.7%, demonstrating consistently strong performance.

Which is better: Viterbi vs single-iteration BCJR?

- BCJR computes soft per-bit posterior probabilities and takes into account the full forward-backward information. As a result, the per-bit MAP estimates from BCJR usually perform at least as well as, and often slightly better than, the Viterbi decoder, which only outputs the single most likely path. Moreover, these soft outputs are particularly useful for iterative decoding schemes like Turbo.
- Viterbi is optimal in terms of the most likely path but does not guarantee optimal per-bit decisions. Consequently, for bitwise error rates, it can be slightly worse than BCJR in some cases.

Experiment: BCJR vs BCJR→Viterbi

Goal. We compare two decoding approaches on 100 testcases:

1. **BCJR (MAP):** Use the BCJR algorithm to compute per-bit a posteriori probabilities and then make hard decisions based on the per-bit MAP.
2. **BCJR → Viterbi:** Take the BCJR per-bit MAP decisions, construct synthetic systematic observations corresponding to these MAP bits, and then run Viterbi on the synthetic systematics along with the original parity stream. This setup effectively tests the impact of "feeding" BCJR outputs as hard information into Viterbi.

Motivation. This experiment aims to determine whether enforcing path consistency via Viterbi after BCJR can improve the bit-error rate (BER) compared to using BCJR per-bit MAP decisions alone. While BCJR optimizes for bitwise accuracy, Viterbi enforces a globally consistent path. In practice, combining BCJR outputs with Viterbi does not necessarily reduce the BER.

Procedure. For each testcase:

1. Initialize the inference object: `infer = Inference(testcase)`.

2. **BCJR per-bit MAP:** Compute the a posteriori probabilities using BCJR and take the per-bit MAP decision:

$$\text{bcjr\_map\_bits}[t] = \arg\max_{b \in \{0,1\}} P(\text{bit}_t = b \mid \text{observations})$$

3. **Construct synthetic systematics:** At each time step t, select the channel level that is most likely given the corresponding BCJR MAP bit.

4. **Viterbi after BCJR:** Run the Viterbi algorithm on the synthetic systematics along with the original parity stream to produce a path-consistent bit sequence.

5. Compare the outputs from both methods against the ground truth and record the number of bit errors.

Pseudocode.

```
for testcase in dataset:
  infer = Inference(testcase)
  post = bcjr(infer.sys, infer.p1, P, None, trellis)
  bcjr_map = [argmax(post[:,t]) for t in 0..T-1]
  sys_synth = [ argmax(P[bcjr_map[t]]) for t in 0..T-1 ]
  vit_after = viterbi(sys_synth, infer.p1, P, trellis)
  compute errors for bcjr_map, vit_after
aggregate statistics (mean/std/min/max BER)
```

Metrics. For each decoding method, we calculate the bit errors per testcase and the corresponding BER (errors / T). These are then aggregated over all testcases to obtain the mean, standard deviation, minimum, and maximum error counts.

Method	Mean BER	Std BER	Min Err	Max Err
BCJR_MAP	0.032100	0.021902	9	72
BCJR_then_Viterbi	0.036200	0.025337	10	85

Table 2: Comparison of bit-error-rates (BER) for BCJR-based decoding strategies on 100 testcases.

Interpretation.

- BCJR per-bit MAP achieves lower mean BER than the BCJR → Viterbi approach because it directly estimates per-bit posterior probabilities.
- Passing BCJR hard decisions into Viterbi slightly increases the BER. This is due to Viterbi enforcing a globally consistent path, which can sometimes flip bits that BCJR had correctly estimated.
- In practice, if the primary goal is to minimize BER, either BCJR per-bit MAP or iterative Turbo decoding is the preferred choice.

Conclusion. The results confirm that BCJR per-bit MAP provides better BER performance than applying Viterbi on BCJR-derived hard decisions. Enforcing path consistency via Viterbi after BCJR MAP does not improve and may slightly worsen the decoding performance.

References

[1] D. Koller and N. Friedman. *Probabilistic Graphical Models: Principles and Techniques*. MIT Press, 2009. https://books.google.co.in/books?id=7dzpHCHzNQ4C

[2] Z. Ghahramani and M. I. Jordan. Factorial Hidden Markov Models. Technical Report AIM-1561, MIT Artificial Intelligence Laboratory, 1997. https://mlg.eng.cam.ac.uk/zoubin/papers/fhmmML.pdf

[3] Massachusetts Institute of Technology. Lecture 8: Inferences on trees: Sum-product algorithm. 6.438 Algorithms for Inference, Fall 2014. https://ocw.mit.edu/courses/6-438-algorithms-for-inference-fall-2014/1faaccb44f78c4f4e99c6814842082a0_MIT6_438F14_Lec8.pdf

[4] R. Das, Z. Liu, and D. Gupta. Lecture 13: Variational inference – loopy belief propagation. Scribe notes for 10-708: Probabilistic Graphical Models, Spring 2014, Carnegie Mellon University. https://www.cs.cmu.edu/~epxing/Class/10708-14/scribe_notes/scribe_note_lecture13.pdf

[5] K. P. Murphy, Y. Weiss, and M. I. Jordan. Loopy belief propagation for approximate inference: An empirical study. arXiv preprint arXiv:1301.6725, 2013. https://arxiv.org/pdf/1301.6725

[6] MIT OpenCourseWare. Lecture 7: Viterbi decoding. 6.02 Introduction to EECS II: Digital Communication Systems, Fall 2012. https://ocw.mit.edu/courses/6-02-introduction-to-eecs-ii-digital-communication-systems-fall-2012/f398fa4a366439301b3d17e45e028952_MIT6_02F12_lec07.pdf

[7] L. Bahl, J. Cocke, F. Jelinek, and J. Raviv. Optimal decoding of linear codes for minimizing symbol error rate. *IEEE Transactions on Information Theory*, 20(2):284–287, 1974. https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=1055186&tag=1